WATERLOO
ENGINEERING

WATERLOO
ENGHACKS

Software Competition

Competition Information Package

Winter 2026
EngHacks
March 7- 8, 2026

Table of Contents

1. Software Specific Rules & Guidelines
    a. Original Code Requirement
    b. Hardware Usage
        i. Guidelines
        ii. Acceptable Hardware Usage Examples
        iii. Not Permitted / Out of Scope Examples
    c. Repository Usage
2. Background Information
3. Problem Statement
4. Functional Requirements
5. Submission Deliverables
6. Presentation
    a. Presentation Deliverables
    b. Presentation Rules
    c. Additional Judging Information
7. Rubric

Software Specific Rules & Guidelines

1. **Original Code Requirement**
   - 1.1. All code submitted must be newly written during the EngHacks competition period.
   - 1.2. Reuse of pre-existing codebases, past projects, or previously developed assignments is not permitted.
   - 1.3. Use of publicly available libraries, frameworks, APIs, and documentation is allowed, provided they are properly cited and not the primary implementation of the solution.
     - 1.3.1. **Where to cite:** Include citations in a dedicated References/Citations section in the project README. Inline comments in code are recommended when directly adapting external logic.
     - 1.3.2. **What to include:** Resource name, source/organization, link, and brief description of its use.

2. **Hardware Usage**
   - 2.1. Guidelines
     - 2.1.1. Teams are permitted to use hardware components as part of their project, provided that the hardware serves as a supplementary interface to a software-based solution.
     - 2.1.2. Hardware will **not** be supplied by EngHacks. Teams are responsible for sourcing, configuring, and maintaining any hardware they choose to use.
     - 2.1.3. Only commercially available hardware is allowed (e.g., laptops, phones, VR headsets, or sensors). Custom-built hardware or hardware where functionality, logic, or innovation is primarily hardware-based is not allowed and may result in disqualification.
     - 2.1.4. Projects that rely primarily on hardware rather than software may be deemed out of scope for the Software Competition and may result in disqualification. The criteria for determining whether a project is out of scope are outlined in Sections 2.2 and 2.3.
     - 2.1.5. Teams may contact competition directors if they are unsure whether a specific hardware use is acceptable. The determination of acceptable hardware usage is at the discretion of the Software Competition Directors, Angela and Doris.
     - 2.1.6. Any questions regarding hardware boundaries should be resolved **before** the project submission to avoid disputes during evaluation.
   - 2.2. Acceptable Hardware Usage Examples
     - 2.2.1. VR/AR Headsets – Headset is input/output only.
     - 2.2.2. Mobile Phones/Tablets – Using GPS, accelerometer, camera, or microphone to collect data.

2.2.3. IoT Sensors – Reading temperature, motion, or humidity data for dashboards or analytics. Hardware provides data; software handles analysis, visualization, and decisions.

2.2.4. Commercial Game Controllers – Hardware serves as an input device; the software drives all game mechanics.

2.2.5. Laptops/Standard Peripherals – Running software, coding environments, or displaying dashboards. Hardware is the baseline; software innovation remains primary.

2.3. Not Permitted / Out of Scope Examples

2.4. Projects where core functionality relies on custom-built hardware (e.g., robotics where movement logic is in circuits, not software).

2.5. Physical devices that cannot function or be evaluated without hardware (e.g., mechanical inventions with minimal software logic).

2.6. Projects where hardware drives the innovation (e.g., creating a new sensor array or custom controller as the main project).

2.7. Projects resembling the Integrated Category (heavy hardware dependency with minimal software contribution).

3. Repository Usage

3.1. The following rules are only applicable to teams submitting their code through the use of a public repository link.

3.2. Pushing/merging code to the main branch of the public repository **after** the deadline will be penalized by Competitor Rule 7.2. Submissions will be checked prior to presentations and teams will be informed if penalizations apply.

3.3. Teams may continue to create issues/PRs after the deadline, as long as they are not merged to main nor presented during the presentation.

3.4. Teams may continue to push code to side branches after the deadline, as long as they are not merged to main nor presented during the presentation.

3.5. Penalization is calculated based on Competitor Rule 7.2 using the last commit/change time as shown on the public repository management service.

Background Information

The Faculty of Engineering at the University of Waterloo is home to one of the most dynamic, fast-paced, and innovative engineering communities in the world. Students balance hands-on learning, challenging technical concepts, experiential education, and co-op opportunities that allow them to apply engineering principles in real industry settings. This environment cultivates creativity, resilience, and problem-solving at an exceptional level.

With such a rich and active academic experience, engineering students regularly engage with a wide range of tools, systems, and platforms, from course materials and design labs to co-op resources, project collaboration tools, and campus services. As the program continues to grow in depth and scope, opportunities emerge to enhance how students access information, manage their commitments, and navigate their academic and professional journeys.

Waterloo Engineering is known for encouraging students to build solutions that not only solve problems, but meaningfully improve the community around them. This year's EngHacks Software Challenge invites competitors to do exactly that, to identify an area where technology can enhance student life and create an impactful, well-designed solution that contributes to the student experience.

Problem Statement

Engineering students at the University of Waterloo engage in a rigorous, collaborative, and innovation-driven environment. They navigate courses, design projects, labs, co-op terms, team-based activities, and campus-wide services throughout the academic year. While these experiences are foundational to the Faculty's commitment to experiential learning, they also involve a variety of tools, processes, and systems that students interact with on a daily basis.

As Waterloo Engineering continues to evolve, opportunities arise to streamline, enhance, or reimagine aspects of student life through software. These improvements can take many forms: making information easier to access, improving communication, supporting academic success, simplifying logistics, increasing inclusivity, or helping students balance academic, professional, and personal responsibilities.

Your challenge is to design and build a software tool, application, or system that **meaningfully improves some aspect of student life for engineering students at the University of Waterloo**. Your solution should address a clear need or opportunity, whether it relates to academics, organization, collaboration, wellness, productivity, or campus engagement. Solutions must be primarily software-driven, though teams may incorporate hardware as a supporting element where appropriate.

Teams may consider:

1. How do engineering students interact with their academic, co-op, and extracurricular environments?

2. Where do friction points or inefficiencies naturally occur?

3. How can thoughtful software design reduce complexity or enhance the student experience?

4. What solution could have practical, lasting value for engineering students?

Your project does not need to solve every problem. Instead, it should focus on one specific area, delivering a meaningful, usable solution that demonstrates creativity, strong engineering design, and relevance to the Waterloo Engineering community.

Unique solutions are awarded.

Functional Requirements

Teams must design and implement a software-based solution that satisfies the following functional requirements:

1. **Functional Code**
    - The final code should be at minimum functional. A judge should be able to reproduce the demo locally if so desired.

2. **Clear Problem Definition**
    - The solution must address a clearly identified problem or opportunity relevant to engineering students at the University of Waterloo.
    - The problem should be articulated concisely and supported by reasonable assumptions or observations about student needs.

3. **Primary Software Focus**
    - The core functionality of the project must be software-driven (e.g., web app, mobile app, desktop application, system, or platform).
    - Any hardware used must act only as a supplementary interface and not the primary driver of the solution.

4. **Usability and User Experience**
    - The solution should be intuitive, easy to navigate, and designed with the end user in mind.

- Interfaces should clearly communicate functionality and minimize unnecessary complexity.

5. **Relevance to Waterloo Engineering**
    - The solution must be clearly tailored to the Waterloo Engineering context (academics, co-op, student life, or campus services).
    - Generic solutions with no clear connection to the community may be scored lower.

Submission Deliverables

Each team must submit the following deliverables before the end of the competition period:

1. **Final Project Submission**
    - A working version of the software solution (e.g., deployed app, runnable project, or executable demo).
    - Source code submitted via a shared repository or compressed file.

2. **Project Documentation**
    - A README or document explaining:
        - The problem being solved
        - Key features and functionality
        - Technologies used
        - Instructions for running or accessing the project
        - Citations

3. **Optional Presentation Slides**
    - A slide deck summarizing the project, suitable for judging and audience presentation may be included but are not required..

4. **Optional Supplementary Materials**
    - Diagrams, design sketches, or short demo videos may be included but are not required.

Software Presentation Deliverables

Teams will engage in a **7 minute presentation** period with a panel of judges. Teams may choose how to allocate their time, however, teams are strongly encouraged to present their solution for the first **5** minutes, followed by a **2** minute Q&A period.

More details of the demo requirements are listed below. Parts of the presentation should be shared equally between team members to score full points.

Your team may discuss elements of your design including, but not limited to:

- **Solution Statement:** What is the issue being solved and why is it important?
- **Decision Making:** What choices lead to the finalized design?
- **Technological Rationale:** Are there performance constraints/considerations?
- **User Centered Design:** How does your solution demonstrate user centered design?

Software Presentation Rules

The **7 minute** timer begins immediately when teams are called into the judging room. Teams will **NOT** be given extra time for late arrivals, or to set up demos and/or supplementary material. It is expected that teams come prepared 5 minutes early with all the necessary components ready to go.

Any demo code, slides, or supplementary materials must be submitted as a deliverable by the appropriate deadline. The use of unsubmitted materials is **strictly prohibited** and violations will result in an **immediate disqualification**.

A timer will be visible during the presentation, teams are expected to be solely responsible for the timing of their presentation. Teams are given **10 seconds** of grace time past their allocated presentation time, where time-based penalties do not apply. There is **3 minutes** of buffer time (time exceeding the allocated 7 minutes) between teams. The use of buffer time for presentation purposes is penalized as follows:

1. **-1%** for every +5 seconds
2. Immediate cutoff at +2:30 minutes with a **-50%** penalty

Additional Judging Information

1. During the Presentations & Judging stage, the judges will be in their respective rooms for the duration of this stage. Teams should be ready with their presentations 5 minutes before their time slot next to their assigned judging room. Teams will be given directions on when to enter the room for judging. Teams will present their design during their slot.
2. After the presentation period, the presenting team shall leave immediately.

Rubric

Criteria	Description	Points	
Problem Definition		**/25**	
Relevance	Does the application address the needs or challenges faced by students and staff in Waterloo Engineering?	**5**	
Practicality	Is the problem scoped correctly? Is the design practical for students and staff in Waterloo Engineering?	**10**	
Clarity	Is the problem and proposed solution clearly defined? Are all claims sufficiently justified?	**10**	
Design Quality		**/50**	
Functionality	Assess the completion, reliability, complexity, and functionality of the demonstrated solution. • ***(20-25 points)*** *Reliable implementation with extra features. No bugs present. Edge cases clearly addressed.* • ***(15-20 points)*** *Reliable implementation with additional features. No bugs present.* • ***(10-15 points)*** *Additional features present with almost no bugs. Only the happy path is covered.* • ***(5-10 points)*** *Working demo present with few bugs. Minimal features available.* • ***(0-5 points)*** *Non-functional or large number of bugs.*	**25**	
Scalability	Is the design easily scalable? Has the team considered scalability?	**5**	
UI	Evaluates the interface for aesthetics and cohesiveness. Areas of evaluations include, but are not limited to, colors, themes, fonts, and layouts.	**5**	
UX	Assess the overall user experience of the interface. Areas of evaluations include, but are not limited to, interactivity, accessibility, intuitiveness, and convenience.	**15**	
Presentation		**/25**	
Communication	Evaluation of the team's ability to cohesively and clearly communicate design intent.	**5**	
Design Decisions	Clear explanation of design choices, including functionality, engineering principles, and technology use.	**15**	

	• ***(13-15 points)*** *Thorough justification for each design choice. Design process clearly documented.* • ***(8-12 points)*** *Thorough explanation with no gaps.* • ***(4-7 points)*** *Clear explanation with minor gaps.* • ***(0-3 points)****: Limited explanation or unclear process.*		
Teamwork	Cooperation and division of tasks within the team.	**5**	
Bonus			
Innovation	Original and unique solutions are awarded. Fully original (never seen before) solutions are awarded the full 10 points.	**up to +10**	
Responsivity	Interfaces with size/device responsiveness are awarded. Ie. phone and tablet responsiveness totals +3 points. Teams are responsible for demonstrating these cases during the presentation	**+1.5 per case**	
Total Score	**/100**		